

 UNIVERSIDAD DON BOSCO	UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERIA ESCUELA DE COMPUTACIÓN
CICLO 2	Materia: Programación Orientada a Objetos (POO404) Guia de Laboratorio No.3 Tema: Programacion orientada a objetos (POO) Parte 1

I. OBJETIVOS.

Que el estudiante:

- Conozca la importancia de la implementación de métodos en Java.
- Implemente en Java a los conceptos fundamentales de la programación orientada a objetos.

II. INTRODUCCIÓN.

Modulos de programas en Java

Hay 2 tipos de módulos en Java: **métodos** y **clases**. Para escribir programas en Java, se combinan nuevos métodos y clases escritas por el programador, con los métodos y clases “**preempaquetados**”, que están disponibles en la interfaz de programación de aplicaciones de Java (también conocida como la API de Java o biblioteca de clases de Java) y en diversas bibliotecas de clases. La API de Java proporciona una vasta colección de clases que contienen métodos para realizar cálculos matemáticos, manipulaciones de cadenas, manipulaciones de caracteres, operaciones de entrada/salida, comprobación de errores y muchas otras operaciones útiles.

Las clases de la API de Java proporcionan muchas de las herramientas que necesitan los programadores. Las clases de la API de Java forman parte del Kit de desarrollo de software para Java 2 (J2SDK), el cual contiene miles de clases preempaquetadas.

Los métodos (también conocidos como funciones o procedimientos en otros lenguajes de programación) permiten al programador dividir un programa en módulos, por medio de la separación de sus tareas en unidades autónomas; también conocidas como métodos declarados por el programador. Las instrucciones que implementan los métodos se escriben sólo una vez, y están ocultos de otros métodos.

Promoción de argumentos

Otra característica importante de las llamadas a métodos es la promoción de argumentos (o coerción de argumentos); es decir, forzar a que se pasen argumentos del tipo apropiado a un método. Por ejemplo, un programa puede llamar al método **sqr** de **Math** con un argumento entero, inclusive cuando el método espera recibir un argumento doble. (Como Programar en Java, 2016)

Paquetes de la API de Java

La palabra clave **package** permite agrupar clases e interfaces. Los nombres de los paquetes son palabras separadas por puntos y se almacenan en directorios que coinciden con esos nombres.

Por ejemplo, los ficheros siguientes, que contienen código fuente Java:

Applet.java, AppletContext.java, AppletStub.java, AudioClip.java

Contienen en su código la línea:

```
package java.applet;
```

Y las clases que se obtienen de la compilación de los ficheros anteriores, se encuentran con el nombre `nombre_de_clase.class`, en el directorio:

```
java/applet
```

Sentencia import

Los paquetes de clases se cargan con la palabra clave **import**, especificando el nombre del paquete como ruta y nombre de clase (análogo a `#include` de C/C++). Se pueden cargar varias clases utilizando un asterisco.

```
import java.Date;  
import java.awt.*;
```

Si un fichero fuente Java no contiene ningún package, se coloca en el paquete por defecto sin nombre.

Paquetes Java

El lenguaje Java proporciona una serie de paquetes que incluyen ventanas, utilidades, un sistema de entrada/salida general, herramientas y comunicaciones. En la versión actual del JDK, los paquetes Java que se incluyen son:

- `java.applet`

Este paquete contiene clases diseñadas para usar con applets. Hay una clase `Applet` y tres interfaces: `AppletContext`, `AppletStub` y `AudioClip`.

- `java.awt`

El paquete `AbstractWindowingToolkit (awt)` contiene clases para generar widgets y componentes GUI (Interfaz Gráfico de Usuario). Incluye las clases `Button`, `Checkbox`, `Choice`, `Component`, `Graphics`, `Menu`, `Panel`, `TextArea` y `TextField`.

- `java.io`

El paquete de entrada/salida contiene las clases de acceso a ficheros: `FileInputStream` y `FileOutputStream`.

- `java.lang`

Este paquete incluye las clases del lenguaje Java propiamente dicho: `Object`, `Thread`, `Exception`, `System`, `Integer`, `Float`, `Math`, `String`, etc.

- `java.net`

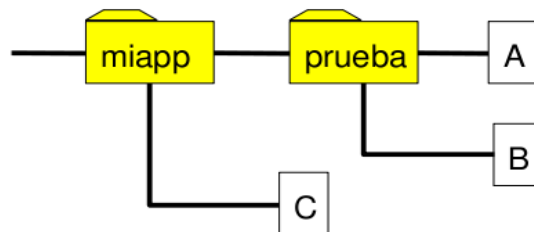
Este paquete da soporte a las conexiones del protocolo TCP/IP y, además, incluye a las clases Socket, URL y URLConnection.

- java.util

Este paquete es una miscelánea de clases útiles para muchas cosas en programación. Se incluyen, entre otras, Date (fecha), Dictionary (diccionario), Random (números aleatorios) y Stack (pila FIFO). Un paquete puede estar situado dentro de otro paquete formando estructuras jerárquicas. Ejemplo:

miapp.prueba.A

- ⤴ Java obliga a que exista una correspondencia entre la estructura de paquetes de una clase y la estructura de directorios donde está situada.
- ⤴ Las clases miapp.prueba.A, miapp.prueba.B y miapp.C deben estar en la siguiente estructura de directorios:



Programacion Orientada a Objetos

Esta tecnología utiliza clases para encapsular (es decir, envolver) datos (atributos) y métodos (comportamientos). Por ejemplo, el estéreo de un auto encapsula todos los atributos y comportamientos que le permiten al conductor del auto seleccionar una estación de radio, o reproducir cintas o CD's. Las compañías que fabrican autos no fabrican los estéreos, sino que los compran y simplemente los conectan en el tablero de cada auto. Los componentes del radio están encapsulados en su caja.

Clase

- ⤴ Conjunto de datos (atributos) y funciones (métodos) que definen la estructura de los objetos y los mecanismos para su manipulación.
- ⤴ Atributos y métodos junto con interfaces y clases anidadas constituyen los miembros de una clase.

Declaración

```
[modificadores] classNombreDeClase{  
  //Declaración de atributos  
  //Declaración de métodos  
  //Declaración de clases anidadas e interfaces  
}
```

- ⤴ **Instancia de una clase.**

Para su uso es necesaria la declaración, la instanciación y la inicialización del objeto.

```
class Empleado{
    //declaracion de campos
    longidEmpleado = 0;
    String nombre = "SinNombre";
    double sueldo = 0;
}
```

^ **Declaración**

```
Empleado e; // declara a una instancia/objeto llamada (e) de la clase Empleado
```

^ **Instanciación**

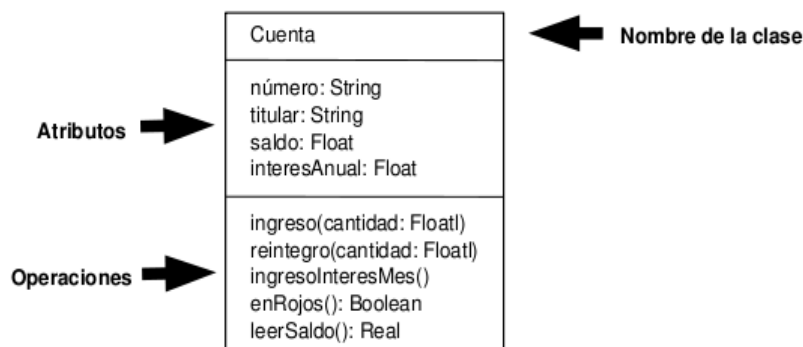
```
e = new Empleado(); //cree una instancia de la clase Empleado
```

^ **Se puede resumir en una única instrucción:**

```
Empleado e = new Empleado();
```

Las clases de objetos representan conceptos o entidades significativos en un problema determinado.

Una clase describe las características comunes de un conjunto de objetos, mediante dos elementos:



- **Atributos (o variables miembro, variables de clase):** describen el estado interno de cada objeto.
- **Operaciones(o métodos, funciones miembro):** describen lo que se puede hacer con el objeto, los servicios que proporciona

El *encapsulamiento* permite a los objetos ocultar su implementación de otros objetos; a este principio se le conoce como ocultamiento de la información. Aunque los objetos pueden comunicarse entre sí, a través de interfaces bien definidas (de la misma forma que la interfaz de un conductor para un auto incluye un volante, pedal del acelerador, pedal del freno y palanca de velocidades), no están conscientes de cómo se implementan otros objetos. Por lo general, los detalles de implementación se ocultan dentro de los mismos objetos. Evidentemente es posible conducir un auto de forma efectiva sin conocer los detalles sobre el funcionamiento de los motores, transmisiones y sistemas de escape.

En los lenguajes de programación por procedimientos (como C), la programación tiende a ser *orientada a la acción*. Sin embargo, la programación en Java está orientada a objetos. En los lenguajes de programación por procedimientos, la unidad de programación es la **función** (en Java las funciones se llaman **métodos**). En Java la unidad de programación es la *clase*. Los objetos eventualmente se *instancian* (es decir, se crean) a partir de

estas clases, y los atributos y comportamientos están encapsulados dentro de los *límites* de las clases, en forma de campos y métodos.

Los programadores, al crear programas por procedimientos, se concentran en la escritura de funciones. Agrupan acciones que realizan cierta tarea en una función y luego agrupan varias funciones para formar un programa. Evidentemente los datos son importantes en los programas por procedimientos, pero existen principalmente para apoyar las acciones que realizan las funciones.

Los *verbos* en un documento de requerimientos del sistema que describen los requerimientos para una nueva aplicación ayudan a un programador, que crea programas por procedimientos, a determinar el conjunto de funciones que trabajarán entre sí para implementar el sistema.

En contraste, los programadores en Java se concentran en la creación de sus propios tipos de referencia, mejor conocidos como *clases*. Cada clase contiene como sus *miembros* a un conjunto de campos (variables) y métodos que manipulan a esos campos. Los programadores utilizan esas clases para instanciar objetos que trabajen en conjunto para implementar el sistema.

Encapsulamiento en Java

En Java, los modificadores de acceso son palabras reservadas que se utilizan para controlar la visibilidad o accesibilidad de clases, métodos, variables, constructores y otros miembros de una clase.

Estos modificadores son fundamentales para implementar el encapsulamiento, un principio clave de la programación orientada a objetos, que ayuda a proteger los datos y a controlar el acceso a ellos.

Los modificadores de acceso permiten definir el alcance de la visibilidad de los elementos de una clase, lo que ayuda a controlar cómo se puede interactuar con ellos desde otras partes del código y promueve la seguridad y el orden en la estructura de un programa.

Los 4 modificadores de acceso principales en Java son:

public:

Un miembro con el modificador public es accesible desde cualquier clase, sin importar el paquete al que pertenezca.

private:

Un miembro con el modificador private solo es accesible dentro de la misma clase en la que se declara.

protected:

Un miembro con el modificador protected es accesible dentro de su propio paquete y también por subclases de otras clases, incluso si están en diferentes paquetes.

Sin modificador (o modificador default/package):

Si no se especifica ningún modificador, el miembro se considera "default" o "package-private". Esto significa que es accesible dentro de su propio paquete, pero no desde fuera de él.

Los distintos modificadores de acceso quedan resumidos en la siguiente tabla:

Modificadores de acceso

	La misma clase	Otra clase del mismo paquete	Subclase de otro paquete	Otra clase de otro paquete
public	X	X	X	X
protected	X	X	X	
default	X	X		
private	X			

III. PROCEDIMIENTO



INFORMACIÓN:

Constructor:

Es un método especial en Java empleado para inicializar valores en Instancias de Objetos, a través de este tipo de métodos es posible generar diversos tipos de instancias para la Clase en cuestión; la principal característica de este tipo de métodos es que llevan el mismo nombre de la clase.

Sobrecarga de métodos y de constructores:

La *sobrecarga de métodos* es la creación de varios métodos con el mismo nombre, pero con diferentes firmas y definiciones. Java utiliza el número y tipo de argumentos para seleccionar cuál definición de método ejecutar.

Java diferencia los métodos sobrecargados con base en el número y tipo de argumentos que tiene el método y no por el tipo que devuelve.

PARTE 1: Definición de clase e instancia de objetos

Descargue los archivos incluidos en los recursos complementarios de este practica 3.

Abra el archivo **POOGuia3.jpg**, el cual describe un diagrama UML con los paquetes, clases y sus miembros a desarrollar en este procedimiento.

1. Cargue la aplicación IntelliJ-IDEA y cree un proyecto Java con el nombre **POOGuia3**.
2. Tenga visible el archivo con el diagrama UML de Clases, ya que cada miembro indicado ahí se ira implementado en su proyecto Java.
3. Para iniciar, dentro de la carpeta **src** del proyecto, cree a los paquetes indicados en el diagrama de clases.
4. Dentro del paquete **misclases**, cree a la Clase **Persona**. En el código generado por el IDE, confirme que incluye al inicio a la sentencia `package misclases;` que indica el paquete dentro del cual se definirá esta clase.
5. Al inicio de la clase, incluya la referencia **`import javax.swing.*;`** con la cual importara a todas las clases del paquete `javax.swing`.
6. Proceda a redactar la siguiente definición de miembros para esta clase.

Importante: Compare la sintaxis del código java con la sintaxis utilizada en el UML:

```
//declaracion de Atributos
private String nombre;
private String apellido;
```

```

private int edad;

//definicion de Metodos

//Metodos Constructores: Se utiliza al instanciar el objeto
public Persona(){ //Constructor por defecto (sin parametros)
    nombre="Rafael";
    apellido="Torres";
    edad=23;
}

//Sobrecarga de Metodo Constructor: Constructor con parametros
public Persona(String nom,String apell,int edad){
    this.nombre=nom;
    this.apellido=apell;
    this.edad=edad;
}

//Permite definir datos a los atributos
public void ingresoDatos()
{
    //utiliza cuadros de dialogo para solicitar valores de campos
    nombre=JOptionPane.showInputDialog("Ingrese el Nombre");
    apellido=JOptionPane.showInputDialog("Ingrese el Apellido");
    edad=Integer.parseInt(JOptionPane.showInputDialog("Ingrese su edad"));
}

//Permite imprimir los valores de los atributos
public void mostrarDatos(){
    System.out.println("Su nombre es: "+nombre);
    System.out.println("Su Apellido es: "+apellido);
    System.out.println("Su edad es: "+edad);
    System.out.println("*****");
}

```

7. Para continuar, tal como se indica en el UML, dentro del paquete **aplicacion** cree a la clase **Principal**. Confirme la aparición de la sentencia *package*, que referencia el paquete **aplicacion** donde se ubica esta nueva clase.
8. Dentro de esta clase **Principal**:
 - Redacte al inicio a la sentencia **import javax.swing.JOptionPane;** para así importar a la clase **JOptionPane** del paquete *javax.swing*.
 - Además, según el diagrama UML, la clase **Persona** a utilizar se encuentra en el paquete **misclases** distinto al paquete **aplicacion** donde está definido el método **main**. Por lo tanto, use otra sentencia **import** para importa a esta clase del paquete **misclases**, así:

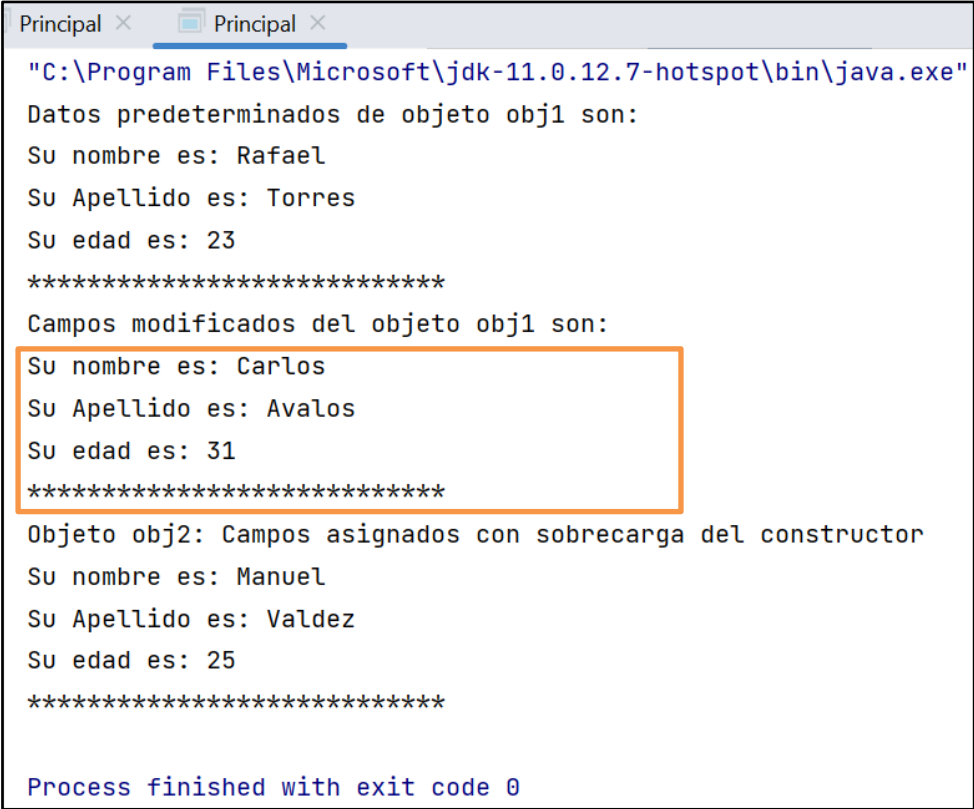
import misclases.Persona;

9. Dentro de la clase, proceda a crear el método estático llamado **parte1** indicado en el UML. Luego, dentro de este método, redacte la siguiente definición:

```
Persona obj1=new Persona();//Declara e instancia al objeto obj1
//Declara e instancia del objeto p2, usando la sobrecarga del Constructor
Persona obj2=new Persona("Manuel", "Valdez", 25);
//Llamamos a el método mostrar datos de obj1
System.out.println("Datos predeterminados de objeto obj1 son:");
obj1.mostrarDatos();
//Cambiamos valor a los atributos de obj1
obj1.ingresoDatos();
System.out.println("Campos modificados del objeto obj1 son:");
//Llamamos a el método mostrardatos de obj1
obj1.mostrarDatos();
//Llamamos a el metodomostrardatos de obj2
System.out.println("Objeto obj2: Campos asignados con sobrecarga del
constructor");
obj2.mostrarDatos();

System.exit(0);
```

10. Ahora cree el método estático de inicio (**main**). Dentro de este método, invoque al método **parte1**.
11. Ejecute la clase **Principal** y observe el resultado en consola. Desde los cuadros de dialogo, proceda a ingresar los datos para modificar los valores de los atributos del objeto **obj1**.
- Dependiendo de lo que usted ingrese en la instancia **obj1**, los resultados serán similares a los mostrados en la siguiente imagen:



```
Principal x Principal x
"C:\Program Files\Microsoft\jdk-11.0.12.7-hotspot\bin\java.exe"
Datos predeterminados de objeto obj1 son:
Su nombre es: Rafael
Su Apellido es: Torres
Su edad es: 23
*****
Campos modificados del objeto obj1 son:
Su nombre es: Carlos
Su Apellido es: Avalos
Su edad es: 31
*****
Objeto obj2: Campos asignados con sobrecarga del constructor
Su nombre es: Manuel
Su Apellido es: Valdez
Su edad es: 25
*****
Process finished with exit code 0
```

PARTE 2: Encapsulamiento: Definición de Propiedades

12. En el ejemplo anterior, la única forma de modificar valores de los campos de un objeto es solamente en 2 momentos: al crearlo o desde dentro de un método publico de la clase.

Ahora se vera como acceder a los mismos de manera individual y restringir la modificación de sus valores utilizando encapsulamiento, gracias a la creación de *Metodos de Propiedad*.

13. De acuerdo con el UML, dentro del paquete **misclases** cree a la clase llamada **Tiempo**.

14. Antes de la definición de la clase, use la sentencia **import** para utilizar a la clase `java.text.DecimalFormat`

15. Redacte la siguiente definición para los miembros de esta clase Tiempo.

```
// Declaración de clase Tiempo: almacena hora en formato de 24 horas
//Atributos/Campos
public int hora; // rango aceptado: 0 - 23
public int minuto; // 0 - 59
public int segundo; // 0 - 59

//metodo para inicializar campos de los objetos
private void InicializarCampos(){
    hora = 0;
    minuto=0;
    segundo=0;
}

// El constructor de Tiempo1 inicializa cada variable de instancia en cero;
// se asegura de que cada objeto Tiempo1 inicie en un estado consistente
public Tiempo()
{
    InicializarCampos(); //fija hora predeterminada (00:00:00 h)
}

// Establecer un nuevo valor de hora utilizando hora universal;
public void establecerHora( int h, int m, int s )
{
    //actualiza elementos de hora segun parametros recibidos
    hora = h;
    minuto = m;
    segundo = s;
}

// Convierte hora (en formato de Hora universal) y lo retorna como String
public String aStringUniversal()
{
    DecimalFormat dosDigitos = new DecimalFormat( "00" );
    return dosDigitos.format( hora ) + ":" +
        dosDigitos.format(minuto) + ":" + dosDigitos.format(segundo);
}

// convertir a String en formato de hora estándar
public String aStringEstandar()
{
    DecimalFormat dosDigitos = new DecimalFormat( "00" );
```

```

        return ( (hora == 12 || hora == 0) ? 12 : hora % 12 ) + ":" +
            dosDigitos.format(minuto) + ":" + dosDigitos.format(segundo) +
            ( hora <12 ? " AM" : " PM" );
    }

```

16. Retorne a la definición de la clase **Principal**. Debido a que la clase **Tiempo** a comprobar esta ubicado en el paquete **misclases**, usted debería agregar otra sentencia **import**, así:

```
import misclases.Tiempo;
```

Pero, debido a que todas las clases a probar en esta parte del procedimiento estarán en un mismo paquete, mejor ubique la sentencia *import* ya existente que referencia a la clase **misclases.Persona** y modifíquela con asterístico (*), así:

```
import misclases.*; //referencia a todas las clases del paquete
```

17. Dentro de la clase, cree a otro método estático bajo el nombre **parte2** y redacte ahí al siguiente código:

```

//declara a objeto hora1 de la clase Tiempo y lo instancia
Tiempo hora1 = new Tiempo(); // llama al constructor de clase Tiempo

// Invoca a metodos del objeto hora para retornar hora almacenada
// en formato estandar y de 24 h.
String salida = "Hora inicial es:\n *hora universal: " +
    hora1.aStringUniversal() + "\n *hora estándar: " +
    hora1.aStringEstandar();
// cambia hora almacenada en el objeto hora
hora1.establecerHora( 18, 27, 6 ); // hora 18:27:06 h
salida += "\n\nLa nueva hora universal es: " +
    hora1.aStringUniversal() +
    "\ny en forma estándar es " + hora1.aStringEstandar();

hora1.hora = 9; // modifica valor del campo hora
hora1.minuto = 52; // modifica valor del campo minuto
salida += "\n\nHora estandar modificada: " + hora1.aStringEstandar();

JOptionPane.showMessageDialog(null, salida,
    "Prueba de clase Tiempo",JOptionPane.INFORMATION_MESSAGE);

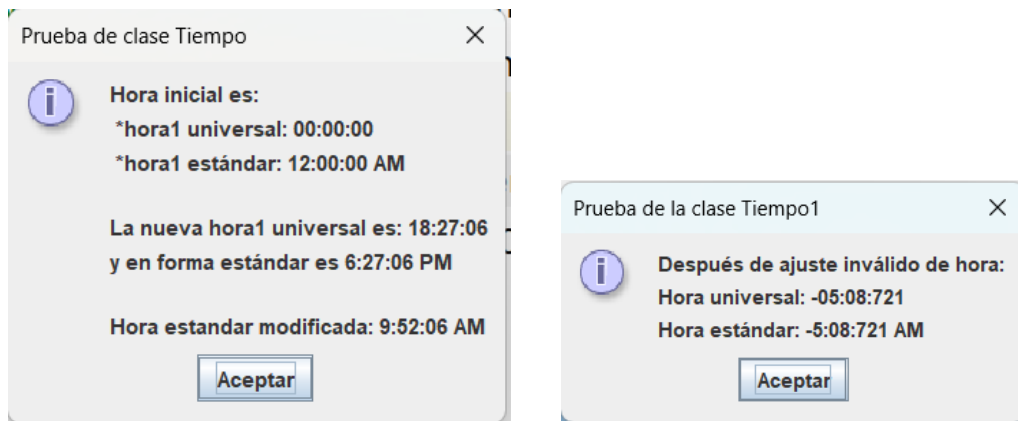
//Establece hora incorrecta: -05:08:721 h
hora1.establecerHora( -5, 8, 721 );
salida = "Después de ajuste inválido de hora: " +
    "\nHora universal: " + hora1.aStringUniversal() +
    "\nHora estándar: " + hora1.aStringEstandar();

JOptionPane.showMessageDialog( null, salida,
    "Prueba de la clase Tiempo1", JOptionPane.INFORMATION_MESSAGE );

System.exit(0);

```

18. Ignore la llamada al método **parte1** y ahora invoque al método **parte2**.
19. Proceda a ejecutar a la clase **Principal**. Observe que la clase **Tiempo** no verifica los valores recibidos, es decir, acepta cualquier valor entero como hora, minutos y/o segundos, por lo que es insegura, no confiable.



20. Por lo tanto, se necesita implementar alguna forma de verificar los valores que usuario intenta asignar y/o modificar en sus campos.

Aquí es donde se aplica el **encapsulamiento de campos**, bloqueando el acceso publico a cada campo y creando por cada campo a 2 metodos de acceso publico, denominados *Metodos de Propiedad*, uno para leer/recuperar (get) el valor actual y el otro para asignar/escribir (set) su valor.

21. Retorne a la definición de la clase **Tiempo**.
22. Para aplicar el encapsulamiento de los campos de esta clase, comience por modificar la visibilidad de cada campo existente de la clase **Tiempo**, aplicando la palabra **private** a la declaración de cada campo, así:

```
private int hora; // rango aceptado: 0 - 23
private int minuto; // 0 - 59
private int segundo; // 0 - 59
```

Por un momento, cambie a la vista del código de **main**. Confirme que los campos **hora** y **minuto** ya no son accesibles desde el objeto **hora1**. Retorne a la definición de la clase **Tiempo**.

Aclaraciones sobre el encapsulamiento de campos:

Los creadores de Java sugieren que para encapsular un campo privado llamado **nombrecampo**, la variable atributo se escriba en minúscula. Luego, los nombres de los métodos de lectura (get) y escritura (set) deben cumplir estas sintaxis:

getNombrecampo **setNombrecampo**

Observe que se convierte a mayúscula la inicial del nombre de la variable del campo de la clase y a este identificador se le antecede las palabras **get** y **set**.

Por ej. Si la variable de un atributo se llamara **montoventa**, los métodos de propiedad se llamarían **getMontoventa** y **setMontoventa**.

Luego, el método `get` para un campo, utiliza la sentencia `return` para retornar el valor del campo

Y para el método `set`, el nuevo valor que quiere asignarse a un campo se recibe en un parámetro. Para el ejemplo del campo `montoventa`, la definición de los métodos `getMontoventa` y `setMontoventa` serian los siguientes:

<pre>public double getMontoventa() { return this.montoventa; }</pre>	<pre>public void setMontoventa(double montoventa) { this.montoventa = montoventa; }</pre>
--	---

23. Continuando con el procedimiento, como 2da etapa del encapsulamiento ubique el cursor al final de la clase **Tiempo** y defina los siguientes *métodos de propiedad* públicos para acceder al campo privado **hora**:

```
public int getHora(){ //metodo GET
    return hora; //retorna valor de campo hora
}

public void setHora(int hora) //metodo SET
{
    //verifica que nuevo valor sea apropiado
    if(hora>=0 && hora<24)
        this.hora = nuevahora; //modifica valor de campo
}
```

24. Para probar la escritura/modificacion del campo `hora`, retorne a la clase **Principal** y ubique la línea que presenta error de acceso al campo **hora**. Reemplace el acceso al campo `hora` (que ya es privado, inaccesible) por la llamada al método **setHora**, enviando como argumento el valor 9 a asignar, así:

```
hora1.setHora(9); // modifica solo a la hora almacenada
```

25. Retorne al código de la clase **Tiempo**.

Ahora vera como el IDE puede ayudarle a encapsular al resto de campos (**minuto**, **segundo**) de su clase **Tiempo**.

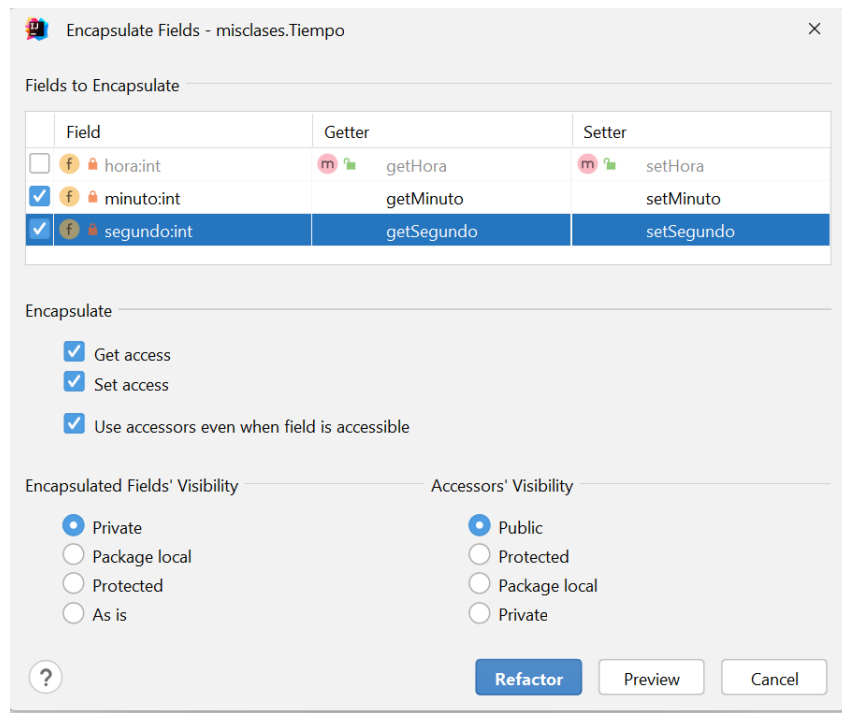
26. Para encapsular al par restante de campos, localice la línea donde declara al campo **minuto** y haga clic secundario sobre su nombre (**minuto**).

Del menú contextual, seleccione la opción *Refactor->Encapsulate Fields*.

De la lista de campos para encapsular, marque a **minuto** y **segundo**.

Verifique que la visibilidad de los campos sea *private* y de los accesoros sea *public*.

Haga clic en botón **Refactor**.



27. Verifique el cambio que hizo el IDE sobre la declaración de los campos y el uso posterior de estos campos elegidos.

Por ej. Haga un seguimiento minucioso del uso del campo **minuto**.

Primero, dentro de la clase **Tiempo**, su visibilidad es ahora **private** y sus métodos de propiedad (accesores **get** y **set**) creados los encuentra al final del código de su clase **Tiempo**. Y por último, observe como se le asigna un valor a este campo; hoy se invoca a su método **setMinuto**. cámbiese al metodo **parte2** y vera que la asignación se hace igual.

28. Debido a que el encapsulamiento del campo **hora** no lo hizo con el asistente del IDE, localice cada uso del campo **hora** y cuando este sea asignado directamente (**hora = ...**), reemplace esta asignacion por la llamada a su método **set** correspondiente (**setHora(...)**). Haga estos reemplazos, tanto en la clase **Tiempo** como en el metodo **parte2**.

29. Ahora implementara la seguridad del acceso al campo **hora**. Dentro de la clase **Tiempo**, localice la definición del método **setHora** y agregue la siguiente toma de decisiones para aceptar horas solamente en el rango (0-23):

```
public void setHora(int hora) //metodo SET
{
    //verifica que nuevo valor sea apropiado (solo 0 y 23 h)
    if(hora>=0 && hora<24)
        this.hora = nuevahora; //modifica valor de campo
}
```

30. De manera similar, localice al método **setMinuto** y agregue la siguiente toma de decisiones para verificar que el campo **minuto** sea modificado solamente si el parámetro esta en el rango (0-59):

```

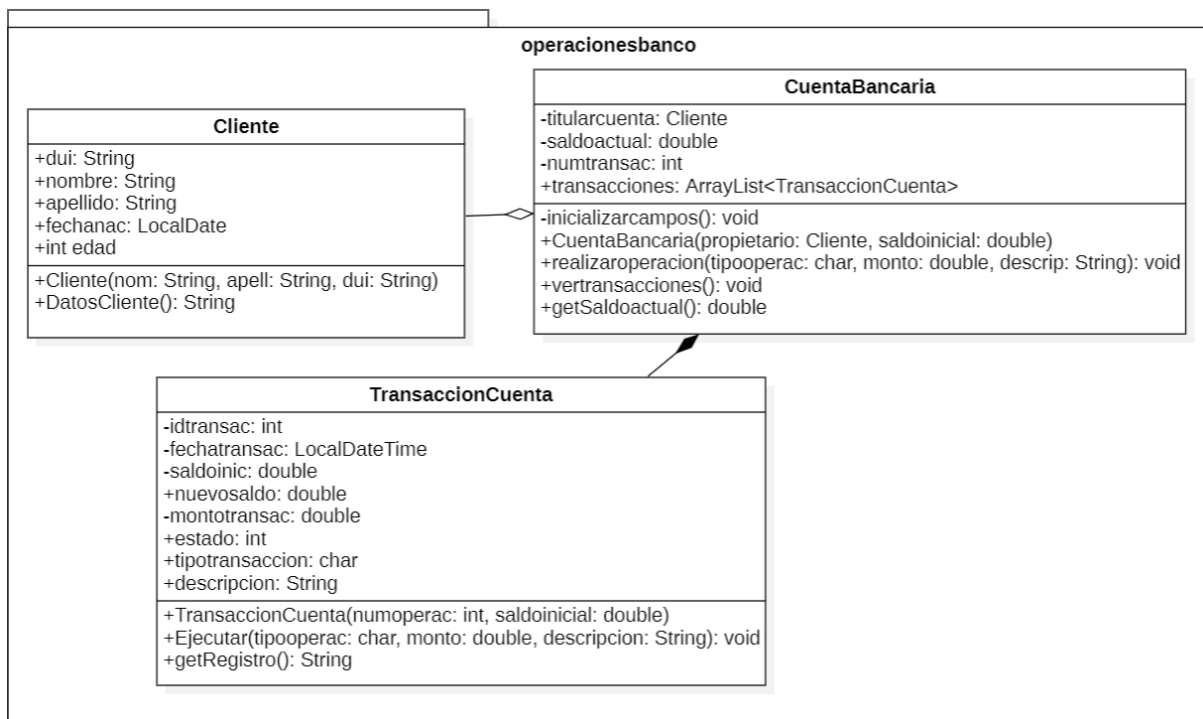
public void setMinuto(int minuto) {
    if(minuto>=0 && minuto<60)
        this.minuto = minuto;
}

```

31. Tome de modelo los cambios hechos para verificar a la hora y a los minutos, para modificar la definición del método **setSegundo** para permitir que el valor aceptado para los segundos este en el rango (0-60).
32. Ejecute nuevamente el programa. Verifique cuando el objeto hora1 permite el cambio de hora, minutos y/o segundos. Si un valor no es aceptado, no modifica el valor actual del campo privado.
33. Guarde los cambios hechos hasta en su proyecto.

PARTE 3: Otro ejemplo de Encapsulamiento

34. Dentro de la carpeta `src`, cree otro paquete bajo el nombre `operacionesbanco` y luego cree en su interior al esquema de clases java indicado en el diagrama UML proporcionado:



35. De acuerdo con las asociaciones binarias indicadas en el diagrama de clases anterior, primero se deben redactar a las clases **Cliente** y **TransaccionCuenta**, ya que sus instancias definen a campos de la clase **CuentaBancaria**.

36. Digite la siguiente definición a la clase **Cliente**:

```
public class Cliente {
    //Campos
    public String dui;
    public String nombre;
    public String apellido;

    //Metodos
    //metodo constructor
    public Cliente(String nom, String apell, String dui){
        this.nombre = nom;
        this.apellido = apell;
        this.oui=oui;
    }
    public String DatosCliente(){
        //ej. de cadena que retorna:
        // Alfredo Ruiz (oui: 4175838-6)
        String datoscliente = String.format("%s %s (oui: %s)",
            this.nombre,this.apellido, this.oui);
        return datoscliente;
    }
}
```

37. Retorne a la clase **Principal**. Para utilizar ahí a todas las clases del nuevo paquete **operacionesbanco**, incluya al inicio a la sentencia **import operacionesbanco.***;

Ademas, importe a la clase **java.util.Scanner**.

38. Cree un metodo estático llamado **parte3**. Luego, redacte el siguiente código en su interior:

```
//Declara a objeto Iris de la clase Cliente
//envia argumentos requerido por su metodo constructor
Cliente Iris = new Cliente(
    "Iris","Bonilla","0858786-6");
System.out.println("Datos del objeto Iris son:");
System.out.println(Iris.DatosCliente());
```

39. Localice la definición del metodo estático **main**. Convierte en comentario a la llamada del metodo **parte2** y agregue una llamada al metodo **parte3**.

40. Ejecute la aplicación y verifique que obtiene el siguiente resultado:

```
Principal x
"C:\Program Files\Microsoft\jdk-11.0.12.7-hotspot\bin\java.e
Datos del objeto Iris son:
Iris Bonilla (oui: 0858786-6)
```


41. Abra la definición de la clase **TransaccionCuenta** y redacte a los siguientes campos:

```
private int idtransac; //num. correlativo de transaccion
private double montotransac; //monto de operacion a ejecutar

// (Año, mes, día) y (hora:minuto) de realizacion de transaccion
private LocalDateTime fechatransac;

private double saldoinic; //saldo antes de ejecutar transaccion
double nuevosaldo; //saldo al completar transaccion
char tipotransaccion; // 'd': deposito, 'r': retiro

String descripcion; // resumen corto de transaccion realizada

//estado de transaccion realizada.
// 0: exitosa, 1: saldo insuficiente, 2: monto negativo,
// -1: operacion no reconocida
int estado;
```

Nota: Verifique que al escribir la declaración del objeto fechatransac, el IDE le incluye al inicio de la clase a una sentencia **import** para importar a la clase **java.time.LocalDateTime**.

42. Ahora digite a los siguientes métodos:

```
//metodo constructor
public TransaccionCuenta(int numoperac, double saldoinicial){
    idtransac = numoperac;

    this.saldoinic = saldoinicial;
    this.nuevosaldo = saldoinicial;
}

public void Ejecutar(char tipooperac, double monto, String descripcion){
    estado = 0; //asume que transaccion se ejecutara con exito
    //del SO obtiene la fecha y hora de transaccion a realizar
    fechatransac = LocalDateTime.now();
    this.tipotransaccion = tipooperac;
    this.descripcion = descripcion;

    //evalua si operacion no puede realizarse
    if(monto<0){
        estado = 2; //monto negativo, no completo transaccion
        return; //cancela transaccion
    }
    //verifica si no hay saldo para retirar monto
    switch (tipooperac){
        case 'r': case 'R': //intenta retirar monto
            if(monto> saldoinic)
            {
                estado = 1; //no hay saldo para retirar monto solicitado
                return; //cancela transaccion
            }
    }
```

```

        //ejecuta retiro
        this.montotransac = monto;
        nuevosaldo = saldoinic - monto;
        break;
    case 'd': case 'D': //ejecuta deposito en cuenta ahorro
        this.montotransac = monto;
        nuevosaldo = saldoinic + monto;
        break;
    default:
        estado = -1; //no reconoce la operacion solicitada
    } //fin switch tipooperac
}

public String getRegistro(){
    String registro, titulooperac = "";
    //define formato elementos de la fecha para registrar transaccion
    DateTimeFormatter formatofecha =
        DateTimeFormatter.ofPattern("yyyy-MM-dd hh:mm:ss");
    String fecha = fechatransac.format(formatofecha);

    switch (this.tipotransaccion){
        case 'd': case 'D':
            titulooperac = "Deposito";
            break;
        case 'r': case 'R':
            titulooperac = "Retiro";
            break;
    }
    registro =
        String.format(
            "%4d.|%20s| %-16s |%-25s |$%10.2f |$%10.2f|",
            idtransac, fecha, titulooperac, descripcion, montotransac,
            nuevosaldo);

    return registro;
}

```

Nota:

Al declarar e instanciar al objeto **formatofecha**, el IDE le importara automáticamente a la Clase `java.time.format.DateTimeFormatter`. Verifique la sentencia *import* creada al inicio del código.

43. Dentro del paquete **operacionesbanco**, abra el archivo de la clase **CuentaBancaria**.

Luego, redacte la siguiente definición para esta clase:

```

// campos
private Cliente titularcuenta;
private double saldoactual;
private int numtransac;

//coleccion de tipo ArrayList de objeto de la clase TransaccionCuenta
private ArrayList<TransaccionCuenta> transacciones;

```

```

//metodos
private void inicializarcampos() {
    titularcuenta = null;
    numtransac = 0;
    saldoactual = 0;
    transacciones = new ArrayList();
}

//metodo constructor
public CuentaBancaria(Cliente propietario, double saldoinicial) {
    //en parametro 1, recibe a objeto de clase Cliente, que sera su Titular
    inicializarcampos();
    this.titularcuenta = propietario;
    //crea instancia de la clase TransaccionCuenta
    TransaccionCuenta transacTemp = new TransaccionCuenta(++numtransac, 0);
    //Para abrir la cuenta de banco, ejecuta transaccion (deposito)
    //con el saldo inicial
    transacTemp.Ejecutar('d', saldoinicial, "apertura de cuenta");
    this.saldoactual = transacTemp.nuevosaldo;
    transacciones.add(transacTemp); //registra transaccion correcta
}

public void realizaroperacion(char tipooperac, double monto, String descrip)
{
    //prepara transaccion
    TransaccionCuenta transacTemp =
        new TransaccionCuenta(numtransac +1, saldoactual);
    transacTemp.Ejecutar(tipooperac,monto,descrip);
    switch (transacTemp.estado)
    {
        case 1:
            System.out.printf(
                "ERROR: Saldo de cuenta insuficiente para retirar $ %.2f\n", monto);
            break;
        case 2:
            System.out.printf(
                "ERROR: Monto $ %.2f requerido no puede ser negativo\n", monto);
            break;
        default:
            numtransac++;
            this.saldoactual = transacTemp.nuevosaldo;
            transacciones.add(transacTemp); //registra transaccion correcta
            System.out.println("Transaccion ejecutada con exito ");
            break;
    }
}

public void vertransacciones(){
    //imprime lista de transacciones realizadas sobre la cuenta actual
    String titulos;

    System.out.printf("\n* Titular de Cuenta: %s\n",
        titularcuenta.DatosCliente());
    System.out.printf("* Saldo actual: $ %.2f\n",saldoactual);
}

```

```

        if(transacciones.size()==0)
            System.out.println("Aun sin transacciones realizadas");
        else{
            titulos = String.format(
                "%4s.|%-20s| %-16s |%-25s |%-10s  |%-11s|",
                "id","fecha","Tipo transaccion","Descripcion","Monto",
                "Saldo");
            System.out.println(titulos);
            //muestra registro de transacciones sobre la cuenta
            for(TransaccionCuenta cuenta:transacciones){
                //recorre colección ArrayList con todos los objetos
                //de transacciones realizadas sobre la Cuenta
                System.out.println(cuenta.getRegistro());
            }
        }
    }

    public double getSaldoactual() {
        return saldoactual;
    }
}

```

Nota: Verifique que al escribir la declaración del objeto transacciones, el IDE le incluyó al inicio de la Clase a una sentencia **import** para importar a la clase **java.util.ArrayList**.

44. Retorne a la clase **Principal** y redacte la siguiente definición al metodo **parte3**:

```

//declara e instancia a objeto c1 basada en la clase CuentaBancaria
//como argumento, envia el objeto Iris como propietario de la cuenta
// y con un saldo de apertura de $20
CuentaBancaria c1 = new CuentaBancaria(Iris,20);

//brinda argumentos estaticos para hacer 2 transacciones sobre la Cuenta
ahorro
c1.realizaroperacion('d',100.2,"premio de loteria");
c1.realizaroperacion('r',300,"viaje a italia");

//brinda argumentos dinamicos (dados por usuario) para ejecutar 3
transacciones mas
Scanner lapiz = new Scanner(System.in);
char tipoperacion;
double monto;
String motivo;
for(int p=1;p<=3;p++) //permite realizar 3 operaciones distintas
{
    System.out.printf("\nDatos de Operacion No.%d a realizar:\n",p);
    System.out.print(
        "Digite letra d para hacer Deposito o letra r para Retirar: ");
    //extrae 1er caracter del texto dado por usuario
    tipoperacion = lapiz.nextLine().charAt(0);
    System.out.print("Digite monto para hacer la operacion: $ ");
    monto = Double.parseDouble(lapiz.nextLine());
    System.out.print("¿Cual es la motivo de la operacion? ");
    motivo = lapiz.nextLine();
}

```

```

//invoca a metodo con argumentos variables
c1.realizaroperacion(tipooperacion, monto, motivo);

System.out.println("Saldo actual: $" + c1.getSaldoactual());
}

//invoca metodo para ver reporte de transacciones validas
// sobre la Cuenta Ahorro
c1.vertransacciones();

```

45. Proceda a ejecutar la aplicación. Compruebe que gracias a 3 Clases distintas (**Cliente**, **CuentaBancaria**, **TransaccionCuenta**), se logra administrar Cuentas de Ahorros y de cada Cuenta, se lleva un registro de sus transacciones aceptadas.
46. Guarde el proyecto y muéstrelo a su instructor.
47. Junto a su instructor, determine el esquema de seguridad de acceso (método *get* y/o metodo *set*) que debería aplicarse a los diferentes campos de las clases que gestionan a las Cuentas de Ahorros.
48. Proceda a encapsular cada campo y nuevamente compruebe que la aplicación funciona correctamente.
49. De los recursos proporcionados, abra el archivo (**POOGuia3completar.mdj**) y redacte a los miembros de las clases del paquete **operacionesbanco**, las cuales incluyan los métodos **getter** y **setter** con los que usted encapsulo a los campos compartidos entre clases.

IV. EJERCICIOS COMPLEMENTARIOS.

En la carpeta **src** del proyecto desarrollado en el procedimiento de esta Practica, cree un paquete y una clase dentro de la misma (con los nombres que usted desee). Dentro de la nueva clase, defina su propio metodo estático **main**.

Dentro de este nuevo metodo **main**, debe elaborar un *Mantenimiento de Cuentas de Ahorros de Clientes*, el cual deberá utilizar a las clases creadas en la Parte 3 de este procedimiento.

El menu principal de su programa constara de las siguientes opciones:

A) Registrar un cliente:

Solicita los datos correspondientes a un nuevo Cliente (objeto de la clase **Cliente**) y almacena el objeto generado de alguna forma en su sistema.

Cada vez que se agregue un cliente (por ej. Raquel Tobar, con dui 5289615), se debe mostrar el listado actualizado de todos los clientes registrados, con un formato similar al siguiente ejemplo, en donde el ultimo cliente siempre será el cliente que acaba de crear dentro de esta opción del mantenimiento:

```
Cientes registrados (4):
# Datos del Cliente
1. Iris Bonilla (dui: 0858786-6)
2. Armando Nieto (dui: 0158224-4)
3. Gilberto Zacarias (dui: 6380044-7)
4. Raquel Tobar (dui: 5289615-0)

-> Presione Enter para retornar al menu
```

B) Abrir una cuenta de ahorro:

Muestra lista de clientes registrados (con el mismo formato del solicitado en literal A)

Luego, el administrador debe elegir a un cliente registrado para crearle una cuenta de ahorro.

Usted decida si en su solucion, el administrador ingresara el num. correlativo (1, 2 ..) del listado de clientes o por el ingreso del dui correspondiente (que es distinto por cada cliente).

Y por ultimo, usted solicitara un monto inicial para la apertura de la cuenta.

Si la cuenta de ahorro se crea correctamente, muestre al administrador las transacciones ejecutadas sobre la nueva cuenta de ahorro, por ej.:

```
* Titular de Cuenta: Raquel Tobar (dui: 5289615-0)
* Saldo actual: $ 20.0
  id.|fecha          | Tipo transaccion | Descripcion          | Monto   | Saldo   |
  1.| 2025-07-15 11:20:47 | Deposito         | apertura de cuenta   | $      20,00 | $      20,00 |

-> Presione Enter para retornar al menu
```

C) Salir de la aplicacion

V. REFERENCIA BIBLIOGRAFICA.

- Como Programar en Java. (2004). 5th ed. Deitel&Deitel.
- Paquetes. (2016). Webtaller.com. Accedido 28 Octubre 2016, de <http://www.webtaller.com/manual-java/paquetes.php>